Karpathy Modelleri FPGA’de · konuşma metni

Nasıl kullanılır: Her başlık bir slayta karşılık gelir; numaralar slayt numarasıdır. Aynı metin sunumda her slaytın konuşmacı notunda da var.

Akış: 1: Konu, sonuçların özeti ve yol haritası. · 4: Hedef: bilgisayarsız çalışan bir FPGA tasarımı ve kısıtları. · 7: Karpathy model ailesi: 9 mimari, stories260K ve TinyStories sürümleri. · 11: Integer-only arithmetic: quantization, requant, softmax, norm ve tablolar. · 18: PyTorch modelinden microcode’a ve memory image’larına. · 21: Bütün modelleri çalıştıran microcoded ortak engine: zoo_core. · 26: Kart sistemi: generation loop, integer sampling, VGA ekranı ve on-board check. · 31: Low-bit weight’ler: Larq, LittleBit ve 1-bit matrix-vector unit. · 37: Verification zinciri ve bulunan bug’lar. · 41: Open-source place and route: Fmax, kaynaklar ve hız. · 46: Ana sorunun cevabı, karşılaştırma, sınırlar ve sonraki adımlar.

---

1. Karpathy modelleri FPGA’de

Hocam merhaba. İlk bölümde minGPT’nin iç yapısını adım adım anlatmış ve bu yapının FPGA açısından potansiyelini değerlendirmiştim. Bu bölümde implementasyonun kendisini sunacağım. Karpathy’nin language model ailesinden dokuz farklı mimariyi ve Karpathy’nin kendi eğittiği stories260K modelini, tek bir FPGA tasarımında, yalnızca integer arithmetic ile ve hiçbir bilgisayara bağlı olmadan çalıştırdık. Sunumda sırasıyla hedefimizi ve kısıtlarımızı, model ailesini, floating point kullanmadan bir language model’i nasıl hesapladığımızı, modeli hardware’in okuyacağı microcode’a çeviren compiler’ı, bütün modelleri çalıştıran ortak engine’i, kartın üzerindeki generation loop’u ve ekranı, hocamın önerdiği LittleBit ve Larq ile low-bit weight’leri, verification yöntemimizi ve son olarak circuit’in chip’e place edildiğinde hangi hızda çalıştığını anlatacağım. Teknik terimleri literatürdeki ve koddaki karşılıklarıyla eşleşsin diye İngilizce bıraktım.

2. Modeller FPGA’de doğru çalışıyor; kalite kaybı yalnızca INT8’e geçişten gelir ve çok küçüktür

Sonuçla başlamak istiyorum ve bunu dört soruya cevap vererek yapacağım. Birinci soru: FPGA modelin hesabını bozuyor mu? Hayır. 18 modelin 18’i, kartın bütününün simülasyonunda Python’daki integer referansla bit-exact sonuç üretti: 11 241 token’ın hepsi, hash, ekran metni ve VGA frame’i birebir aynı. Yani FPGA’nin kendisi hiçbir kayıp eklemiyor. İkinci soru: floating point’ten integer’a geçmek kaliteyi düşürüyor mu? Çok az. Loss, names modellerinde en fazla yüzde 0,3, Karpathy’nin stories260K modelinde yüzde 1,6 artıyor. Hesabın hiçbir adımında floating point kullanmıyoruz. Üçüncü soru: modeller chip’e gerçekten sığıyor mu? 16 model open-source araçlarla place and route’tan geçti ve timing’i karşılıyor; en büyüğü bile LUT’ların yüzde 31’ini kullanıyor. İki TinyStories modeli henüz placement aşamasında takılıyor. Dördüncü soru hız: stories260K, 25 megahertzde saniyede 496 token üretiyor. Aynı modeli Karpathy’nin run.c koduyla Apple M1 CPU’da çalıştırınca saniyede 14 397 token çıkıyor; yani dizüstü bilgisayar mutlak hızda önde, ama token başına harcanan cycle’da FPGA 4,4 kat daha verimli. Bu dört cevabın ayrıntılarını sunum boyunca göstereceğim.

3. Sunum, bir modelin PyTorch'tan chip'e uzanan yolunu dokuz adımda izliyor

Sunumun akışı, bir modelin PyTorch'tan chip'e uzanan yolunu izliyor. Önce hedefimizi ve kartın bize koyduğu kısıtları anlatacağım. Ardından model ailesini tanıtacağım: dokuz mimarinin ortak ve farklı yönleri. Üçüncü bölüm işin matematiği: floating point kullanmadan bir language model'in nasıl hesaplandığı. Dördüncü bölüm compiler: bir PyTorch modelinin hardware'in okuyacağı microcode'a ve memory image'larına nasıl çevrildiği. Beşinci bölüm bütün modelleri çalıştıran ortak engine, altıncı bölüm de engine'in çevresindeki kart sistemi: generation loop, sampling ve ekran. Turuncu başlıklı son üç bölüm projenin bu aşamasında eklenen çalışmalar: hocamın önerdiği LittleBit ve Larq ile low-bit weight'ler, verification yöntemimiz ve circuit'in chip'e place edilmesiyle elde edilen hız.

4. Amaç, eğitilmiş bir language model'i bir bilgisayara bağlı olmadan doğrudan FPGA'nin içinde çalıştırmaktır

Hedefimizi iki yaklaşımı karşılaştırarak anlatmak istiyorum. FPGA hızlandırıcılarında yaygın olan yaklaşım soldaki: bir bilgisayar seri port üzerinden FPGA'ye her token için komut gönderir, FPGA bir hesap adımını yapar, bilgisayar sonucu okur, bir sonraki token'ı kendisi seçer ve metni kendisi ekrana yazar. Bu düzende FPGA yalnızca bir hesaplayıcıdır; dışarıdan bakıldığında model bilgisayarda çalışıyormuş gibi görünür. Biz bu yaklaşımı bilinçli olarak seçmedik. Sağdaki tasarımımızda modelin weight'leri bitstream ile birlikte chip'in BRAM'ine yükleniyor; engine, sampling, token'ın metne çevrilmesi ve ekrana yazılması chip'in içinde gerçekleşiyor ve sonuç doğrudan VGA monitöre gidiyor. Çalışmanın kapsamını üç ilkeyle sınırladık: FPGA yalnızca inference yapıyor, training bilgisayarda; yalnızca Karpathy'nin depolarındaki modelleri kullanıyoruz; verification'ı ise fiziksel kart yerine Mac üzerinde, kartın tamamını simüle ederek yapıyoruz.

5. Hedef kart Nexys A7-100T, CPU'su bulunmayan orta büyüklükte bir Artix-7 FPGA kartıdır

Hedef kartımız Digilent'in Nexys A7-100T kartı. Üzerinde AMD'nin, eski adıyla Xilinx'in Artix-7 ailesinden XC7A100T chip'i bulunuyor: 63 400 LUT, 126 800 flip-flop, 135 adet 36 kilobitlik BRAM, yani yaklaşık 607 kilobyte on-chip memory, ve 240 DSP multiplier. Kartta ayrı bir CPU yok; bu da tasarımı her işin FPGA'de yapılmasına zorluyor. Soldaki şekil kullandığımız input ve output'ları gösteriyor: 100 megahertzlik oscillator clock signal'ini veriyor; VGA output'u metni monitöre yazıyor; 7-segment gösterge hızı gösteriyor; switch'lerle sampling modu, temperature ve seed seçiliyor; button'larla koşular başlatılıyor ve ekran temizleniyor; LED'ler token sayısını ve on-board check'in sonucunu gösteriyor. Bitstream USB-JTAG ile ya da açılışta QSPI flash'tan yükleniyor. Kartta 128 megabyte'lık bir DDR2 memory de var; onu bilinçli olarak kullanmıyoruz ve nedenini bir sonraki slaytta açıklayacağım.

6. Tasarımı dört kısıt belirledi: CPU, floating point ve external memory kullanılmıyor; tek bir clock var

Tasarımı dört kısıt belirledi. Birincisi CPU'nun bulunmaması: bir bilgisayarda birkaç satır Python olan generation loop, sampling, token'ın karakterlere çevrilmesi ve ekrana yazma, burada her biri bir hardware state machine olarak tasarlandı. İkincisi floating point kullanılmaması: chip'teki DSP

block'ları integer multiplier'larıdır; floating point toplama ve çarpma LUT'larla kurulabilir ama çok pahalıdır. Bu yüzden bütün hesabı INT8 ve INT16 ile yaptık; exponential, square root ve division da dahil. Üçüncüsü external memory kullanılmaması. Bu bilinçli bir tercih: model chip içine sığdığı sürece memory delay'i olmuyor ve her işlemin süresi önceden bilinebiliyor. Sayılarla bakarsak, stories260K'nın 260 bin weight'i birer byte olarak yaklaşık 254 kilobyte; 256 token'lık context için KV cache 80 kilobyte. Toplam 607 kilobyte'ın rahatça altında. Karşılaştırma için GPT-2'nin 124 milyon parametreli sürümü 124 megabyte gerektirirdi ve ancak DDR ile çalıştırılabilirdi. Dördüncüsü tek clock: 100 megahertzlik oscillator'dan bir MMCM ile core clock'unu üretiyoruz; VGA'nın pixel clock'unu da ayrı bir clock yerine aynı clock'tan bir enable signal'iyle türetiyoruz. Böylece clock domain geçişi problemi hiç oluşmuyor.

7. Dokuz mimari Karpathy'nin dört deposundan alındı ve aynı ~50 bin parametre bütçesiyle eğitildi

Model ailemiz Karpathy'nin dört deposundan geliyor. İlk beş model makemore'dan: bigram yalnızca son karaktere bakan bir tablo; mlp, Bengio'nun 2003'teki modeli gibi son 16 karakterin embedding'lerini birleştiriyor; rnn ve gru recurrent modeller ve geçmişi bir state vector'ünde taşıyorlar; bow ise geçmişin causal ortalamasını alan basit bir bag of words modeli. Gri satırlar transformer'lar. gpt2, minGPT ve nanoGPT'nin ortak GPT-2 bloğu: LayerNorm, GELU ve öğrenilen pozisyon embedding'leri. microgpt, Karpathy'nin bu yıl yayımladığı yaklaşık 200 satırlık microgpt gist'inden geliyor: parametresiz RMSNorm, ReLU ve hiç bias yok. llama, llama2.c'nin mimarisi: gain'li RMSNorm, SwiGLU, RoPE ve grouped-query attention. nanochat, Karpathy'nin en yeni deposunun temel mimarisi: parametresiz RMSNorm, ReLU kare, RoPE, GQA ve QK-norm. Karşılaştırmanın adil olması için hepsini aynı veri setinde, names.txt'te, ve aynı yaklaşık 50 bin parametre bütçesiyle eğittik; bigram doğası gereği 729 parametrede kalıyor. Transformer'ların hepsi 4 layer, 32 boyutlu embedding ve 16 token'lık context kullanıyor.

8. Mimariler aynı yapı taşlarının farklı birleşimleridir; engine bu yapı taşlarının her birini bir op olarak sunar

Bu dokuz mimari ilk bakışta çok farklı görünse de hardware açısından aynı yapı taşlarının farklı birleşimleridir. Tablo, her modelin microcode'unda hangi op türlerinin gerçekten cycle harcadığını gösteriyor; veriler Verilator profilinden alındı. Her model bir embedding okumasıyla başlıyor; bigram için bu tek op yeterli, çünkü embedding satırı doğrudan logit'leri veriyor. mlp'den itibaren her modelin ana işi GEMV, yani matrix-vector multiplication. Recurrent modeller state vector'ünü taşımak için COPY ve GRUUPD gibi küçük op'lar kullanıyor; gru ayrıca gate'ler için element-wise multiplication, yani MUL kullanıyor. bow kendi causal ortalama op'una sahip. Transformer'larda NORM ile attention ve KV cache ekleniyor; llama ve nanochat'te RoPE, llama'da SwiGLU için yine MUL. Son sütun bir token için microcode'da kaç op bulunduğunu gösteriyor: bigram'da 1, nanochat'te 60, stories260K'da 73. Bu tablonun sonucu şudur: on altı op türünü destekleyen tek bir engine, bütün aileyi çalıştırmaya yetiyor. Activation fonksiyonları ayrı bir unit gerektirmiyor; GEMV'nin output'unda bir LUT ile uygulanıyor.

9. stories260K, Karpathy'nin TinyStories üzerinde eğittiği bir Llama 2 modelidir; diğer mimariler de aynı boyutta eğitildi

Ailenin en önemli üyesi stories260K, çünkü ailede başkasının eğittiği tek model. Karpathy bu modeli llama2.c deposunda, kısa çocuk hikâyelerinden oluşan TinyStories veri setinde eğitip yayımladı. Mimari eksiksiz bir Llama 2: 64 boyutlu embedding, 5 layer, 8 attention head ve 4 KV head, yani grouped-query attention, ve 512 token'lık bir vocabulary. Context 512 token, ancak KV cache BRAM'e sığsın diye kartta 256 ile sınırlıyoruz. Karpathy'nin checkpoint dosyasını doğrudan okuyoruz; float hâli ile INT8 hâli arasındaki fark 1,400'den 1,422'ye, yani 0,022 nat. Sağdaki tablo bizim TinyStories sürümlerimizi gösteriyor: aynı mimarileri aynı veride, stories260K ile aynı boyutta, yaklaşık 260 bin parametreyle eğittik. Bu ölçekte Llama mimarisi en iyi sonucu veriyor, GPT-2 ikinci sırada; microgpt, gru ve nanochat daha geride. Bir uyarı: sağdaki değerler kendi test düzenimizle, stories260K'nınki ise 400 validation hikâyesiyle ölçüldü; bu yüzden iki tabloyu doğrudan karşılaştırmıyorum.

10. Aynı circuit, BRAM'deki modele göre isim ya da hikâye üretir; iki ekran görüntüsü de kartın VGA output'undan alındı

Bunlar kartın VGA output'undan alınmış iki frame. Fotoğraf değiller: kartın bütününü simüle ederken VGA pinlerinden, yani kırmızı, yeşil, mavi ve sync signal'lerinden yakaladığımız frame'ler. Python'daki referans ekranla karşılaştırıldığında aralarında tek bir pixel farkı yok. Soldaki gpt2 modeli names.txt üzerinde eğitildiği için alt alta yeni isimler üretiyor; bu koşuda 196 token, token başına 8 713 cycle. Sağdaki Karpathy'nin stories260K modeli "Once upon a time, there was a little girl named Lily" diye başlayan 256 token'lık bir hikâye yazıyor; token başına 50 397 cycle. Üstteki satır model başlığı; alttaki status line token sayısını, hızı, cycle sayısını, sampling ayarlarını ve on-board check'in sonucunu gösteriyor. "check OK" ifadesi, kartın Python ile aynı token'ları ürettiğini gösteriyor. Status line'daki token/s değeri 50 megahertz varsayımıyla hesaplanıyor; placement bölümünde gerçek clock frequency'sini göstereceğim. İki ekranı da aynı circuit üretiyor; değişen yalnızca BRAM'deki model.

11. Integer arithmetic, çarpma-toplama işlemini tek bir DSP bloğunda tek cycle'da yapar ve modeli dört kat küçültür

Integer arithmeticni iki nedenle seçtik. Birincisi hesap maliyeti. Bir float32 çarpma-toplama dört adım gerektirir: üslerin toplanması ve mantislerin çarpılması, toplanacak sayıyla üslerin hizalanması, toplama ve normalizasyon, son olarak rounding. FPGA'de hazır bir floating point unit bulunmadığı için bu adımların hepsi LUT'larla kurulmak zorunda ve hem pahalı hem yavaş. INT8'de ise ortadaki örnekteki gibi: x ve w vector'lerinin dot product'ı 12 çarpı 18, eksi 7 çarpı eksi 55 ve 30 çarpı 127; toplam 4 411. Her terim bir DSP48 bloğunda tek cycle'da hesaplanıyor: 8 bitlik iki sayı çarpılıyor ve 32 bitlik bir accumulator'a ekleniyor. İkinci neden memory. stories260K'nın weight'leri float32 olarak bir megabyte'ı aşıyor ve chip'in 607 kilobyte'lık BRAM'ine sığmıyor; INT8'de ise 254 kilobyte tutuyor. Bu yaklaşım bir gereklilik getiriyor: her sayı artık bir scale ile çarpılmış bir integer. Scale'lerin nasıl seçildiğini ve integer'lar arasındaki işlemlerin floating point'e dönülmeden nasıl yapıldığını sonraki beş slaytta anlatacağım.

12. Quantization, her gerçel sayıyı bir scale ile çarpılmış bir integer olarak temsil eder

Quantization'ın temel fikri basit: her gerçel sayıyı bir scale ile çarpılmış bir integer olarak temsil ediyoruz. Soldaki örneğe bakalım. Bir weight satırında mutlak değeri en büyük eleman 0,268 olsun. INT8'in pozitif tarafı 127'ye kadar gittiği için scale, 0,268 bölü 127, yani 0,00211 olur. Ardından her weight bu scale'e bölünüp round edilir. 0,0372, 17,63 eder ve 18'e round edilir; geri çevrildiğinde 0,0380 elde edilir ve error 0,0008'dir. Eksi 0,1153, eksi 55 olur. En büyük eleman tam olarak 127'ye karşılık gelir ve error'ı sıfırdır. Çok küçük olan 0,0009 ise 0,43 eder ve sıfıra round edilir. Error hiçbir zaman scale'in yarısını, yani 0,00106'yı aşmaz. Sağdaki formül bunun genel hâli. Burada iki tasarım kararı var. Weight'lerde her output channel'ının, yani her satırın kendi scale'i bulunuyor; böylece büyük değerli bir satır küçük değerli bir satırın resolution'ını bozmuyor. Activation'larda ise per-tensor sabit bir scale kullanıyoruz ve bu scale'i kalibrasyon verisinden, mutlak değerin yüzde 99,99'luk percentile'ından seçiyoruz. Maksimum yerine percentile seçmek nadir uç değerleri kırpıyor ve kalan değerlere daha ince bir resolution kazandırıyor; bu seçimi model ailesi üzerinde ölçerek yaptık.

13. Requant, real-valued scale'i bir integer multiplier ve bir shift ile uygular; hiçbir adımda floating point kullanılmaz

Integer arithmetic'in temel işlemi requant. Bir Linear layer'ın output'unu düşünelim. Weight'ler S_w, input'lar S_x scale'inde integer'lar; integer'ları çarpıp topladığımızda bir accumulator elde ediyoruz ve gerçek output yaklaşık olarak S_x çarpı S_w çarpı accumulator oluyor. Bir sonraki layer ise bu output'u kendi scale'i S_y'de INT8 olarak bekliyor. Dolayısıyla accumulator'ı M ile çarpıp round etmemiz gerekiyor; M de S_x çarpı S_w bölü S_y, yani bir gerçel sayı. Bunu floating point kullanmadan yapmak için M'yi bir integer m_0 ile 2 üzeri eksi sh'nin çarpımı olarak yazıyoruz; m_0 2 üzeri 15 ile 2 üzeri 16 arasında seçildiğinde relative error 2 üzeri eksi 16'dan küçük kalıyor. Üstteki akış, hardware'in yaptığı işi sırasıyla gösteriyor: accumulator'a bias ekleniyor, m_0 ile çarpılıyor, rounding için 2 üzeri sh eksi 1 ekleniyor, sh bit sağa shift ediliyor ve sonuç INT8 range'ine saturate ediliyor. Sağdaki örnekte scale'ler 0,05, 0,00211 ve 0,077; M 0,0013703 çıkıyor ve sh 25 seçildiğinde m_0 45 979 oluyor. Accumulator 5 321 ise çarpım 244 654 259, yarım eklenince 261 431 475, 25 bit shift edilince 7 elde ediliyor. Floating point ile hesaplasaydık 5 321 çarpı M, 7,29 ve round edildiğinde yine 7. Bu yöntem Google'ın 2018'deki integer-only inference çalışmasından alınmıştır.

14. Integer softmax, exponential'ı bir shift ve 256 entry'lik bir lookup table'a ayırır ve head başına yalnızca bir division yapar

Attention'ın merkezindeki softmax'ı integer ile dört adımda hesaplıyoruz; üstteki akış bunu gösteriyor. Birinci adımda en büyük score bulunup her score'dan çıkarılıyor. Softmax bundan etkilenmiyor ama bütün üsler sıfır ya da negatif hâle geliyor, yani hiçbir değer taşmıyor. Fark, scale'leri ve $\log_2 e$'yi içeren bir sabit C ile çarpılıyor. İkinci adımda e üzeri x, 2 üzeri x çarpı $\log_2 e$ olarak yazılıyor. Bu üssün tam kısmı n, 2 üzeri eksi n ile çarpmak demek, bu da bir sağa shift; fractional kısım f ise 256 entry'lik bir lookup table'dan okunuyor. Üçüncü adımda bütün e'ler ve value'larla weighted sum biriktiriliyor. Dördüncü adımda probability'ler tek tek bölünmüyor; en sonda bir kez 2 üzeri 32 bölü E hesaplanıp çarpılıyor, yani her head için tek bir division yapılıyor.

Sağdaki örnekte score'lar 9, 6 ve 2, C ise 92. Fark çarpı C sırasıyla 0, 276 ve 644. 276, 256 çarpı 1 artı 20 olarak ayrışıyor: tablonun 20. satırı okunup bir bit shift ediliyor ve 15 520 elde ediliyor. Toplam 54 018; bölündüğünde probability'ler 0,607, 0,287 ve 0,106 çıkıyor ve float softmax ile üç basamakta örtüşüyor. Aynı LUT kartta bir sonraki token'ı seçen sampler'da da kullanılıyor.

15. Integer LayerNorm ve RMSNorm, vector başına bir square root ve bir division kullanır; output'un overflow olmayacağı ispatlanmıştır

Normalization layer'ları, yani LayerNorm ve RMSNorm, mean, variance, square root ve division gerektiriyor. Bunları integer ile beş aşamada hesaplıyoruz. Önce elemanlar toplanıyor. Ardından her eleman N ile çarpılıp toplamdan çıkarılıyor: bu, N çarpı x eksi ortalama demek, ancak division gerekmiyor ve hiçbir bilgi kaybolmuyor. Bu farkların kareleri toplanıp epsilon'un karşılığı olan sabit ekleniyor. Sonra bu toplamın integer square root'u bit bit, sequential bir circuit ile alınıyor; ardından sabit bir K bu köke bölünüyor. Son aşamada her fark bu tersle çarpılıp shift ediliyor. RMSNorm'da ortalama çıkarılmıyor; aynı unit'te toplam sıfır alınıyor. Bu tasarımın önemli bir garantisi var: Samuelson eşitsizliğine göre her elemanın ortalamadan uzaklığı en fazla sigma çarpı kök N eksi 1'dir. Dolayısıyla normalize edilmiş output'un büyüklüğü veriden bağımsız olarak sınırlı; output formatını bu sınıra göre seçtiğimizde ne kalibrasyon gerekiyor ne de saturation oluşuyor. Üçüncü nokta, LayerNorm'un gain'i ve bias'ı arkasından gelen Linear'ın weight ve bias'ına matematiksel olarak fold ediliyor; hardware yalnızca normalize edilmiş vector'ü hesaplıyor. Maliyete bakıldığında NORM, gpt2'de cycle'ların yüzde 14'ünü, QK-norm nedeniyle nanochat'te yüzde 33'ünü kullanıyor.

16. INT8 input'lu her activation fonksiyonu, 256 entry'lik bir lookup table ile yaklaşıma gerek kalmadan temsil edilir

Activation fonksiyonları için çok basit ama güçlü bir gözlem kullanıyoruz: bir fonksiyonun input'u INT8 ise input yalnızca 256 farklı değer alabilir. O hâlde fonksiyonu 256 entry'lik bir lookup table ile tam olarak temsil edebiliriz; polinom yaklaşımına ya da parça parça doğrusal bir yaklaşıma gerek yok. Soldaki grafik GELU için: mavi çizgi gerçek GELU, turuncu noktalar tablonun değerleri. Noktalar eğrinin tam üzerinde; tek fark, output 127'de doyuyor. Sağdaki tablo birkaç satır: input integer'ı 20 ise gerçek değer 1,0, GELU'su 0,841 ve output scale'ine bölünüp round edilince 34. Bu yöntemle GELU, SiLU, tanh, sigmoid ve ReLU kare, hepsi aynı mekanizmayla yapılıyor; her model hangi fonksiyonu kullanıyorsa compiler o LUT'u üretiyor. Engine'de ayrı bir activation unit yok: LUT GEMV'nin output'una takılıyor; accumulator önce requant ile INT8'e iniyor, sonra LUT'tan geçiyor. Layer başına 256 byte.

17. INT8'e geçişin kaliteye etkisi küçüktür: loss names modellerinde en fazla 0,006, stories260K'da 0,022 nat artar

Bütün bu integer hilelerinin kaliteye bedeli ne? Bu tablo, float model ile hardware ile bit bit aynı çalışan integer programın aynı test verisindeki loss'larını karşılaştırıyor. Çubuklar loss farkı. names modellerinde fark en fazla 0,0055 nat, nanochat'te; bigram ve bow'da pratikte sıfır, hatta bow'da integer model binde üç daha iyi, bu gürültü düzeyinde. En büyük fark Karpathy'nin stories260K modelinde: 0,022 nat. Bu beklenen bir şey, çünkü bu model bizim kalibrasyonumuzu bilmeden, float olarak eğitilmiş ve 5 layer derinliğinde; quantization error'ları layer'lar boyunca birikiyor. Yine de 1,400'e karşı 1,422, yüzde 1,5'lik bir fark. Son sütun başka bir bakış: her adımda en olası token'ın

float ile integer modelde aynı çıkma oranı. Küçük modellerde yüzde 96 ile 100 arası, stories260K'da yüzde 92. Bu farklılıklar genellikle probability'leri birbirine çok yakın iki token arasında oluyor; loss farkının bu kadar küçük olmasının sebebi bu.

18. Compiler, eğitilmiş bir PyTorch modelini altı adımda hardware'in okuyacağı microcode'a ve memory image'larına dönüştürür

Bir modeli hardware'e götüren software tarafı, yani compiler, altı adımdan oluşuyor. Birinci adımda eğitilmiş modeli alıyoruz: kendi eğittiklerimiz için PyTorch dosyası, stories260K için Karpathy'nin llama2.c formatındaki .bin dosyası. İkinci adımda modeli bir ara temsile, IR'a çeviriyoruz: embedding, norm, GEMV, attention gibi op'ların düz bir listesi. Bu listeyi float olarak çalıştırıp PyTorch modeliyle karşılaştırıyoruz; fark 10 üzeri eksi 4'ün altında olmalı. Üçüncü adımda matematiksel olarak eşdeğer dönüşümleri yapıyoruz: norm'un gain ve bias'ını, attention'daki 1 bölü kök d'yi weight'lere katlıyoruz. Dördüncü adım kalibrasyon: örnek verilerle activation'ların scale'lerini seçiyoruz. Beşinci adımda integer programı elde ediyoruz. Bunu çalıştıran Python sınıfı IntRunner bizim golden reference'ımız: hardware, bu programın ürettiği her sayıyı bit bit aynı üretmek zorunda. Altıncı adımda programı ve verileri, Verilog'un readmemh ile okuyacağı memory dosyalarına yazıyoruz; bitstream üretilirken bunlar BRAM'lerin başlangıç içeriği oluyor. Tablo gpt2 için dosyaları gösteriyor: 43 op'luk microcode, 64 bitlik 6 272 weight word'ü, per-channel requant parametreleri ve LUT'lar.

19. Microcode, modelin forward fonksiyonunun op dizisi olarak yazılmış hâlidir; şekil gpt2 ile llama'nın ilk block'unu karşılaştırır

Microcode'un neye benzediğini görmek için iki modelin ilk block'unu yan yana koydum. Bunlar compiler'ın gerçekten ürettiği op'lar. Soldaki gpt2: önce residual buffer R'yi LayerNorm ile normalize edip A'ya yazıyoruz; A'dan üç GEMV ile query, key ve value'yu hesaplıyoruz; key ve value'yu KV cache'e yazıyoruz; attention dört head için output'u B'ye yazıyor; projection GEMV'si sonucu R'ye ekliyor, yani residual bağlantı "artı eşittir" modunda bir GEMV. Sonra ikinci LayerNorm, GELU LUT'u output'a takılı fc GEMV'si ve yine residual'a eklenen bir projection. Sağdaki llama'da yapı aynı, üç fark turuncu: query ve key'e RoPE uygulanıyor, key için yalnız 2 head var çünkü grouped-query attention; ve MLP SwiGLU: w1'in output'u SiLU LUT'undan geçiyor, w3'ün output'uyla element-wise çarpılıyor, sonra w2. Önemli olan şu: iki mimari arasındaki fark birkaç op; geri kalan her şey aynı ve engine ikisini de aynı circuit ile çalıştırıyor. Programın sonunda son norm ve logit'leri üreten head GEMV'si var.

20. En büyük model olan stories260K, BRAM'in yaklaşık %71'ini kullanır ve bunun en büyük kısmını weight'ler oluşturur

Modelin chip'in içine sığması tasarımın temel varsayımıydı; bunu en büyük modelle kontrol edelim. Çubuk, stories260K'nın on-chip memory'sini gösteriyor; çubuğun tamamı 607 kilobyte. En büyük pay weight'lerin: 256 kilobyte. Sonra KV cache, 256 pozisyonluk context için 80 kilobyte. Token embedding'leri ve exp LUT'u gibi table'lar 68 kilobyte; her output channel'ının bias, m_0 ve sh değerleri 28 kilobyte; microcode ve activation LUT'ları birkaç kilobyte. Geriye 170 kilobyte kadar boş yer kalıyor. Yosys synthesis'i bunu 97,5 BRAM bloğu, yani kapasitenin yüzde 72'si olarak tahmin ediyor. Alttaki tablo karşılaştırma: names modelleri yüzde 1 ile 14 arasında; binary stories260K ise

weight'ler 8 kat küçüldüğü için yüzde 41'e iniyor. Önemli bir ayrıntı: weight'ler bitstream'in bir parçası. Verilog'daki readmemh ile okunan .mem dosyaları BRAM'lerin başlangıç içeriği oluyor; kart açılırken FPGA yapılandırılırken weight'ler de yerine yazılmış oluyor. KV cache ve vector buffer'ları ise çalışırken yazılan sıradan RAM'ler.

21. zoo_core, microcode'u sırayla okuyan ve her op'u ilgili compute unit'ine yönlendiren tek bir engine'dir

Bütün modelleri çalıştıran engine'in adı zoo_core. Çalışma şekli bir CPU'ya benziyor ama çok daha basit. Üstteki şerit control flow: engine program memory'sinden bir op word'ü okuyor; bu word'de op türü, kaynak ve hedef buffer'lar, vector boyutları ve weight address'leri var. Op'u ilgili unit'e gönderiyor, unit bitince bir sonraki word'e geçiyor; END op'una gelince bir token'ın hesabı bitmiş oluyor ve logit'ler generation loop'a gidiyor. Soldaki sütun modele ait memory'ler: INT8 weight'ler, 1-bit weight'ler, requant parametreleri, tablolar ve çalışırken yazılan KV cache. Ortada compute unit'leri. Turuncu olanlar matrix-vector multiplication: 8 lane'li INT8 GEMV, 64 lane'li 1-bit GEMVB ve ikisinin ortak kullandığı seri requant ve activation LUT'u. Mavi olanlar transformer'a özgü unit'ler: norm, attention ile KV cache yazımı ve RoPE. Alttakiler küçük yardımcı op'lar: embedding okuma, element-wise multiplication ve recurrent modellerin state işlemleri. Sağda 11 vector buffer'ı var; residual akış INT16, diğerleri INT8. Hepsi tek bir modülde; bir model hangi op'ları kullanmıyorsa o unit'ler synthesis'te atılabiliyor, bunu birkaç slayt sonra göstereceğim.

22. GEMV unit'inde her cycle'da bir input sekiz lane'e broadcast edilir ve sekiz output aynı anda birikir

Hesabın büyük kısmını yapan GEMV unit'ine yakından bakalım. Soldaki şekil bir weight matrix'inin 8 satırlık bir output block'unu ve input vector'ünü gösteriyor. Her cycle'da input vector'ünün bir elemanı, mesela x_3, 8 lane'in hepsine birden broadcast ediliyor. Aynı cycle'da weight memory'sinden 64 bitlik tek bir word okunuyor; bu word'de o input'a ait 8 farklı output'un weight'i yan yana duruyor. Her lane kendi weight'ini x ile çarpıp kendi accumulator'ına ekliyor. Input vector bitince 8 output'un accumulator'ı hazır. Bu düzenin adı output-stationary: output'lar yerinde kalıyor, input'lar akıyor. Weight memory'si tam bu sırayla yazıldığı için address basit bir counter. Block bitince 8 accumulator bir gölge register bankasına kopyalanıyor ve oradan requant unit'ine tek tek akıyor; bu sırada lane'ler bir sonraki 8 output'a başlamış oluyor, yani boşluk yok. Cycle sayısı bu yüzden basit: output sayısının 8'e bölümü çarpı input sayısı. gpt2'nin 32'den 128'e fc layer'ı 16 block çarpı 32, 512 cycle; stories260K'nın 64'ten 172'ye w1 layer'ı 22 block çarpı 64, 1 408 cycle. Bu yüzden bir modelin token başına cycle sayısı önceden, kağıt üzerinde hesaplanabiliyor.

23. Attention unit'inde her geçmiş token, her head için iki cycle ekler; bu nedenle maliyet context uzunluğuyla doğrusal artar

Attention unit dört aşamalı. SCORE aşamasında query, KV cache'teki her key ile çarpılıyor; head boyutu 8 olduğu için 8 multiplier bir key'in dot product'ını tek cycle'da bitiriyor. Score'lar gelirken EXP aşaması onları softmax için 2'nin kuvvetine çeviriyor ve topluyor; bu ikisi örtüşüyor. DIV aşamasında head başına tek bir division yapılıyor. AV aşamasında value'lar attention weight'leriyle toplanıyor, yine 8 lane ile. Sonuç: her geçmiş token, her head'de score için bir, value için bir, yani iki cycle ekliyor. Bunu ölçümle doğrulayabiliriz. stories260K'da ilk pozisyonda token başına 38 652

cycle, son pozisyon 511'de 79 532 cycle. Aradaki fark bölü 511, tam 80,0 ediyor; model de 5 layer çarpı 8 head çarpı 2, yani 80 diyor. Ölçüm ile model birebir aynı. Bu doğrusal büyüme, kartta context'i neden 256 ile sınırladığımızın ikinci sebebi: hem KV cache 80 kilobyte'ta kalıyor hem de en uzun adım yaklaşık 59 bin cycle'da kalıyor.

24. makemore modellerinde cycle'ların neredeyse tamamı GEMV'ye gider; transformer'larda norm ve attention da önemli pay alır

Engine'in cycle'ları nereye harcadığına bakalım; veriler Verilator'daki op bazlı profilden. Turuncu GEMV, koyu mavi norm, açık mavi attention, gri diğer op'lar. makemore modellerinde, yani mlp, rnn, gru ve bow'da, cycle'ların yüzde 94 ile 96'sı GEMV; bunlar neredeyse saf matrix-vector multiplication. Transformer'larda tablo değişiyor: gpt2'de GEMV yüzde 77, norm yüzde 14, attention yüzde 7. nanochat'te norm yüzde 33'e çıkıyor, çünkü nanochat her query ve key'i de normalize ediyor, QK-norm; bu da her layer'da ek norm demek. stories260K'da ise attention yüzde 37 ile ikinci büyük kalem; çünkü burada context 512'ye kadar çıkıyor ve attention önceki slayttaki gibi pozisyonla büyüyor. Bu profil, optimizasyonun nereye yapılması gerektiğini söylüyor: küçük modellerde GEMV paralelliği, uzun context'te attention, nanochat'te norm. Sağdaki sayılar engine'in token başına ortalama cycle sayısı; 9 ile 59 bin arasında.

25. SPECIALIZE ile modelin kullanmadığı unit'ler synthesis'te kaldırılır; makemore modellerinde engine dörtte birine küçülür

Engine bütün op'ları desteklese de her model hepsini kullanmıyor; bigram yalnızca bir tablo okuyor. Bu yüzden bir SPECIALIZE parametresi ekledik. Compiler, her model için hangi op'ların kullanıldığını bir bit maskesine yazıyor; sağdaki kod parçasındaki OPS_USED bu. SPECIALIZE açıkken her unit'in logic'i bu maskenin ilgili bitiyle koşullanıyor; bit sıfırsa o logic sabit olarak kapalı oluyor ve synthesis aracı onu tamamen siliyor. Soldaki çubuklar bunun etkisi: gri, bütün unit'lerle; turuncu, yalnızca kullanılan unit'lerle LUT sayısı. bigram 5 855'ten 1 529'a, mlp 8 094'ten 2 502'ye iniyor; makemore modellerinde engine yaklaşık dörtte birine küçülüyor. Transformer'lar zaten çoğu unit kullandığı için kazanç az: stories260K'da yüzde 9. Kart tasarımında her model kendi uzmanlaşmış engine'iyle synthesize ediliyor. Karşılaştırma için bütün unit'leri içeren ve modeli sonradan yükleyebilen tek bitstream'lik sürüm 9 444 LUT.

26. Engine'in çevresindeki hardware, normalde bir bilgisayarın yaptığı generation loop, sampling, metin ve ekran işlerini üstlenir

Engine tek başına yalnızca "bu token ve bu pozisyon için logit'ler" sorusunu cevaplıyor. Bir language model'i kullanılabilir yapan geri kalan her iş, normalde bir bilgisayarda Python ile yapılıyor. Biz bunları da hardware'e koyduk. Üst sıra control path: switch ve button'lar debounce edilip synchronize ediliyor; generation loop engine'e token ve pozisyonu veriyor, logit'leri alıp sampler'a gönderiyor ve seçilen token'ı bir sonraki adımın input'u yapıyor. Alt sıra output path: detokenizer token numarasını karakterlere çeviriyor, stories260K'da bir token birden fazla karakter olabiliyor; terminal bu karakterleri 80 sütunluk bir metin alanına yazıyor, kelimeleri bölmeden alt satıra kaydırıyor, ekran dolunca scroll ediyor ve en alttaki status line'ı güncelliyor; vga_text de bu metni 640 çarpı 480 resolution'da, 8'e 16 pixel'lik bir fontla monitöre çiziyor. Altta üç ayrıntı: on-board

check, ekran yazımının bir sonraki token'ın hesabıyla paralel yürümesi, yani maliyetsiz olması, ve 7-segment ile LED'ler.

27. Generation loop, engine'i çalıştıran, logit'leri üç kez tarayan ve bir sonraki token'ı seçen bir state machine'dir

Generation loop bir state machine. G_STEP'te engine'e mevcut token'ı ve pozisyonu verip başlatıyor, G_WAIT'te engine'in END op'una gelmesini bekliyor. Sonra logit'leri üç kez tarıyor; turuncu satırlar bunlar. G_MAX en büyük logit'i buluyor; greedy moddaysak seçim bu kadar, en büyük logit'in token'ı doğrudan seçiliyor. Sampling modunda G_SUM her token için 2 üzeri eksi fark çarpı c'yi hesaplıyor, ki bu temperature'lı softmax'ın normalize edilmemiş hâli; aynı anda min-p kesmesini uyguluyor ve toplamı biriktiriyor. G_SEL'de random bir sayıyı toplamla çarpıp bir threshold buluyor ve birikimli toplam bu threshold'u ilk aştığında o token'ı seçiyor. G_PICK seçilen token'ı ekrana gönderiyor, hash'i güncelliyor, pozisyonu artırıyor ve döngü başa dönüyor. Bu taramaların her biri vocabulary boyu kadar cycle sürüyor; toplam ek maliyet yaklaşık 3V. stories260K'da bu 1 500 cycle civarı, token başına toplamın yüzde 3'ü; names modellerinde vocabulary 27 olduğu için 80 cycle kadar.

28. Integer sampling, temperature ve min-p'yi softmax ile aynı LUT ile ve division kullanmadan uygular

Bir sonraki token'ı seçmek, yani sampling, normalde temperature ile bölünmüş logit'lerin softmax'ını alıp o distribution'dan random bir sample çekmek demek. Bunu da integer'a çevirdik. Logit'lerin en büyüğünü çıkarıp farkı c ile çarpıyoruz; c, logit scale'ini, $\log_2 e$'yi ve temperature'ı içeren bir sabit. Sonra attention'daki aynı 2'nin kuvveti LUT'uyla e_k'leri buluyoruz. Min-p kuralı, olasılığı en olası token'ın 32'de birinden küçük olanları atıyor; en büyük e her zaman 2 üzeri 15 olduğu için bu basit bir karşılaştırma. Division'a hiç gerek yok: probability'leri normalize etmek yerine random number'ı toplamla çarpıp 32 bit shift ediyoruz ve birikimli toplamın bu threshold'u ilk aştığı token'ı seçiyoruz. Sağdaki örnek dört token'lı: logit'ler 120, 100, 60 ve 20; scale 0,05, temperature 0,8. Üçüncü ve dördüncü token min-p ile kesiliyor. Kalan iki token'ın probability'leri 0,7775 ve 0,2225; float hesap 0,7773 ve 0,2227 veriyor, binde birden küçük fark. Random number 0,75'e karşılık geliyorsa threshold 31 610 ve ilk token seçiliyor; 0,875'e karşılık geliyorsa ikinci token. Python'daki referans aynı algoritmayı birebir uyguladığı için kartın seçtiği token'lar Python'unkilerle aynı.

29. Token'lar karakterlere çevrilir, kelimeler bölünmeden yazılır ve metin VGA output'unda her frame'de yeniden çizilir

Ekran tarafında üç iş var. Detokenizer, token numarasını karakterlere çeviriyor. Bunun için iki ROM kullanıyoruz: biri her token'ın karakter dizisinin nerede başladığını ve kaç karakter olduğunu, diğeri karakterlerin kendisini tutuyor. names modellerinde her token bir harf; stories260K'da bir token " the" gibi birkaç karakter olabiliyor. Terminal bu karakterleri 80 sütunluk bir metin alanına yazıyor. İlk sürümde kelimeler satır sonunda ortadan bölünüyordu; şimdi bir kelime 80. sütunu aşacaksa yazılmış yarısı alt satıra taşınıyor. Şekilde satır sonlarına bakarsanız hiçbir kelimenin bölünmediğini görürsünüz. Ekran dolunca satırlar dairesel bir buffer'da kayıyor. vga_text, 640 çarpı 480 resolution'da, saniyede 60 frame, her frame'de metin RAM'ini baştan okuyup font ROM'undan pixel'leri çiziyor; üç stage'li bir pipeline. En alttaki status line'daki token/s değerini bir bölücüyle

hesaplıyor ve sayıları bin2bcd modülüyle onluğa çeviriyoruz. Bu frame de pinlerden yakalandı ve referansla pixel pixel aynı.

30. Switch'ler üretimi ayarlar, button'lar koşuları başlatır; LED16 kartın Python ile aynı sonucu ürettiğini gösterir

Kart bir bilgisayara bağlı olmadığı için bütün kontrol switch ve button'lardan. Güç verildiğinde ya da reset'e basıldığında ilk koşu kendiliğinden başlıyor: stories260K bir hikâye, names modelleri 27 isim yazıyor. Orta button yeni bir koşu başlatıyor; random number generator'ın durumu kaldığı yerden sürdüğü için her basışta farklı bir hikâye geliyor. Üst button generator'ı switch'lerden okunan seed ile reseed ediyor, alt button ekranı temizliyor. Switch'lerle greedy ya da sampling seçiliyor, sekiz temperature ve dört min-p değeri arasından seçim yapılıyor; daktilo modu üretimi gözle izlenebilecek bir hıza indiriyor; 7-segment gösterge token/s ya da token başına cycle gösteriyor. Sağdaki kutu on-board check: kart ürettiği her token ile basit bir hash güncelliyor. Python, modeli dışa aktarırken aynı koşuların hash'ini hesaplayıp bitstream'e gömüyor; kart koşu bitince ikisini karşılaştırıyor. RGB LED üretim sırasında mavi, sonuç aynıysa yeşil, farklıysa kırmızı yanıyor. Yani kart, Python ile token token aynı sonucu bulduğunu kendisi söylüyor; simülasyonda 18 modelin hepsinde yeşil.

31. 1-bit weight'ler memory'si sekiz kat küçültür ve çarpma işlemini toplama ile çıkarmaya dönüştürür

Bu bölüm, hocamın önerdiği iki kaynağın, LittleBit makalesinin ve Larq kütüphanesinin, projede nasıl kullanıldığını anlatıyor. Önce motivasyon. Soldaki INT8 örneği daha önce gördüğümüz dot product: üç çarpma. Sağda aynı weight'leri yalnızca sign'larıyla ve tek bir scale ile temsil ediyoruz: artı, eksi, artı. Artı bir ile çarpmak sayıyı olduğu gibi almak, eksi bir ile çarpmak sign'ını çevirmek demek. Yani 12 artı 7 artı 30, 49; sonra bir kez scale ile çarpıyoruz. Matrix-vector multiplication'ın ağır kısmında hiç çarpma kalmıyor ve her weight 8 yerine 1 bit. LittleBit bunu bir adım ileri götürüp weight matrix'ini low-rank binary factor'lara ayırarak weight başına 1 bitin altına iniyor; Larq ise binary ağlar için bir kütüphane ve Bi-Real Net, ternary weight networks gibi bilinen quantizer'ları içeriyor. Kural değişmedi: FPGA'de training yok. Low-bit weight'leri Mac'te quantization-aware training ile öğreniyoruz, FPGA yalnızca sonucu çalıştırıyor. Literatürdeki gibi embedding ve output head INT8 kalıyor.

32. Beş yöntem aynı training düzeniyle karşılaştırıldı; Larq'ın quantizer'ları ve LittleBit PyTorch'ta yeniden yazıldı

Beş yöntemi aynı training düzeniyle karşılaştırdık. int8 engine'in bugünkü formatı ve training gerektirmiyor. int4, standart quantization-aware training ile 4 bit. Ternary ve binary Larq'ın quantizer'ları: ternary, Larq'ın ternary weight networks ayarındaki SteTern'ü, weight'leri eksi a, sıfır ve artı a'ya indiriyor. Binary, Larq'ın Bi-Real Net'ten gelen MagnitudeAwareSign'ı ile her weight'i sign'ına indiriyor ve per-channel ortalama mutlak değeri scale olarak kullanıyor. Turuncu son satır LittleBit: weight matrix'i, iki binary factor ve üç scale vector'ünün çarpımı olarak temsil ediliyor; başlangıç makaledeki Dual-SVID yöntemiyle, rank ve path sayısı ayarlanabiliyor. Bu yöntemlerin hepsinin training'indeki ortak hile straight-through estimator: ileri yönde weight'in sign'ı kullanılıyor, geri yönde ise gradient, sign fonksiyonu yokmuş gibi, ama yalnızca weight eksi bir ile artı bir

arasındaysa geçiriliyor. Training gerçek değerli bir gölge weight'i güncelliyor. Ayrıca float modeli teacher yapıp knowledge distillation kullandık. Larq aslında bir TensorFlow kütüphanesi; quantizer'ları birkaç satırlık fonksiyonlar olduğu için onları belgelenen tanımlarına göre PyTorch'ta yeniden yazdık. Böylece modellerimiz ve integer akışımız hiç değişmedi.

33. Transformer'da binary, geniş input'lu MLP'de LittleBit daha iyi sonuç verir; INT4 her modelde float ile aynı kalitededir

İki modelin sonuçları üzerinden anlatayım; diğer altı model aynı örüntüyü gösteriyor. Mavi noktalar gpt2, turuncular mlp; yatay eksen QAT sonrası test loss, yani solda olan daha iyi; sağdaki sayılar weight başına bit. İlk gözlem: INT4 her iki modelde de float kadar iyi, gpt2'de knowledge distillation sayesinde biraz daha bile iyi. İkinci gözlem: ternary binary'den fazla bit harcıyor ama binary'den iyi değil. Asıl ilginç olan binary ile LittleBit karşılaştırması. gpt2'de binary 1,38 BPW'de 2,029 ile float'a çok yakın; LittleBit aynı ya da daha fazla bitte daha kötü, mesela lb25p2 2,08 BPW'de 2,090. mlp'de durum tersine dönüyor: binary 2,160 ile kötü, LittleBit lb25p2 1,18 BPW'de 2,075 ile çok daha iyi. Fark layer'ların genişliğinden geliyor: gpt2'nin layer'ları 32 genişliğinde, mlp'nin ilk layer'ının input'u ise 256. Bunun neden böyle olduğunu bir sonraki slaytta formülle göstereceğim. Bu bulgu hocamın verdiği araştırma sorusunun cevaplarından biri: LittleBit küçük modellerde ancak geniş layer'larda işe yarıyor.

34. Bunun nedeni, LittleBit'in scale vector'lerinin dar layer'larda bit bütçesinin büyük kısmını kullanmasıdır

Bir önceki slayttaki örüntünün sebebi bu formülde. LittleBit bir layer için iki tür bit harcıyor: binary factor'ların sign bitleri ve 16 bitlik scale vector'leri, yani h, g ve ℓ. İlk terim rank ile, ikinci terim layer'ın boyutlarıyla orantılı. Weight sayısına böldüğümüzde scale'lerin bedeli layer genişliği d ile bir bölü d olarak azalıyor. Sağdaki tablo bunu sayılarla gösteriyor. names modellerindeki 32'ye 32'lik bir layer'da, hiç sign biti saymadan, yalnızca scale'ler weight başına 1 bit tutuyor; bu, binary bir layer'ın toplam 1,5 bitlik bedelinin üçte ikisi. Yani LittleBit'in bit bütçesinin çoğu rank'a değil scale'lere gidiyor; binary ise aynı bütçeyle tam rank'lı bir sign matrix'i kullanıyor. 4096'ya 4096'lık bir LLM layer'ında ise yalnız scale'ler 0,008 BPW; tamamen ihmal edilebilir. Makaledeki 0,1 BPW sonucu tam olarak bu genişlikten geliyor: 4096 kare, rank 86, iki path, 0,0998 BPW. Hocamın sorduğu ilk araştırma sorusunun cevabı bu: küçük modellerde LittleBit'in anlamlı alt sınırını layer genişliği belirliyor ve 64'ün altında binary daha verimli.

35. Engine'e 64 lane'li ve multiplier'sız bir 1-bit GEMV unit eklendi; binary ve LittleBit modelleri aynı unit'te çalışır

Sonuçlara göre hardware kararı şu oldu: engine'e 1-bit weight'lerle çalışan bir matrix-vector unit ekledik, adı GEMVB. 64 bitlik bir weight word'ü, bir input'un 64 farklı output için sign'larını taşıyor; 64 lane aynı anda input'u ya ekliyor ya çıkarıyor. Multiplier yok, DSP kullanılmıyor. Accumulator'lar 20 bit yetiyor, çünkü her terim en fazla 128. Requant ve control logic INT8 GEMV ile ortak; per-channel scale de requant multiplier'ına fold ediliyor. İkinci sorun normalization'ın gain'iydi. Compiler normalde γ'yı arkadaki weight'e katlıyor; ama binary bir matrix çarpı diag γ artık binary değil. Bu yüzden fold edilmiş matrix'i satır scale'i, sign matrix'i ve sütun scale'i olarak ayırıyoruz. Satır scale'i requant'a gidiyor; sütun scale'i ise yeni bir SCALE op'u oluyor ve aynı input'u okuyan q, k ve v için tek

seferde uygulanıyor. Üçüncü kutu: tek path'li LittleBit de aynı unit ile çalışıyor; bir SCALE ve iki GEMVB op'u, ara scale'ler requant'lara fold ediliyor. Unit yalnız modelde GEMVB op'u varsa synthesize ediliyor. İlk sürüm çok LUT tutuyordu, 14 709; accumulator'ları INT8 bankasından ayırıp 20 bite indirince 11 153'e düştü.

36. Kartta low-bit modeller 1,7–3,6 kat hızlıdır ve daha az BRAM kullanır; bunun karşılığında loss artar

Low-bit üç modeli de kart tasarımına koyduk ve diğer modeller gibi bit-exact doğruladık. Tablo sonuçları gösteriyor. Binary gpt2, INT8 gpt2'den 1,7 kat hızlı: token başına 8 713 yerine 4 983 cycle. LittleBit'li mlp 3,6 kat hızlı; mlp'nin ilk layer'ı çok geniş olduğu için 64 lane'li unit'ten en çok o yararlanıyor. Binary stories260K 1,8 kat hızlı ve BRAM kullanımı 97,5 block'tan 56'ya iniyor. Kazancın neden 8 kat olmadığı ortada: yalnızca matrix-vector multiplication'lar hızlanıyor; norm, attention ve sampler aynı kalıyor. Bedelin iki tarafı var. Toplama lane'leri LUT'larla kurulduğu için LUT sayısı yüzde 35 ile 60 artıyor; chip'te bunun için bolca yer var. Asıl bedel kalite: names'te binary 0,03, LittleBit 0,16 nat kaybettiriyor. stories260K'da ise binary 0,64 nat kaybettiriyor; 260 bin parametre bu görev için zaten sınırda ve weight'leri 1 bite indirmek çok ağır geliyor. Loss eğrisi son adımlarda hâlâ düşüyordu; daha uzun bir QAT bu farkı azaltabilir.

37. Verification sekiz aşamalı bir zincir olarak kuruldu; her aşama bir öncekinin çıktısını referans alır

Verification'ı sekiz halkalı bir zincir olarak kurduk; her halka bir öncekinin çıktısını referans alıyor. Gri ilk iki halka software'de: bizim float modelimizin Karpathy'nin koduyla aynı sonucu verdiğini, 10 üzeri eksi 4 toleransla, ve integer modelin kalitesinin korunduğunu, loss farkıyla ölçüyoruz. Turuncu halkalar hardware'in tasarımını doğruluyor: kartın tamamının RTL simülasyonu, random button ve switch testleri ve bilinmeyen değerleri yakalayan 4-state simülasyon. Mavi halkalar araçları doğruluyor: synthesis aracının ürettiği kapı listesinin simülasyonu ve placement'tan sonra timing analysis. Sekizinci halka kartın kendisi olurdu; kararımız gereği bunu yapmıyoruz. Neden bit-exact şart? Çünkü hardware da referans da aynı integer işlemlerini yapıyor; çıktılar yaklaşık değil tam olarak aynı olmak zorunda. Yaklaşık bir karşılaştırma bir bug'ı saklayabilir; bit-exact karşılaştırmada tek bir farklı token bug'ı ortaya koyuyor. Kart olmadan bu neden yeterli? Tasarımda tek clock var ve her şey synchronous. Netlist functionally doğruysa ve timing analysis her path'in bir clock period'una sığdığını gösteriyorsa, chip'teki circuit simülasyondaki gibi davranır. Endüstride de bir chip üretime gitmeden önce bu iki kanıta dayanılır.

38. Tam kart simülasyonunda testbench bir kullanıcı gibi button'lara basar ve yalnızca pinlerden okunabilen değerleri karşılaştırır

Verification'ın merkezindeki test, kartın tamamının simülasyonu. Burada önemli bir tasarım kararı var: testbench tasarımın içine bakmıyor, gerçek bir kullanıcı gibi davranıyor. Reset'i bırakıyor, ilk koşunun bitmesini bekliyor, sonra button'lara basıyor ve yalnızca kartın pinlerinden okunabilecek şeyleri karşılaştırıyor. Soldaki tablo karşılaştırılanları gösteriyor: üretilen her token, on-board check'in sonucu, ekrandaki metnin tamamı, VGA pinlerinden yakalanan bir frame, 7-segment gösterge ve LED'ler; ayrıca her cycle'da bütün pinlerin bir imzası tutuluyor, bu sonraki netlist karşılaştırmasında işe yarıyor. Her model için dört koşu yapılıyor: otomatik sampling koşusu, orta

button ile ikinci bir koşu, greedy modda reseed edilmiş bir koşu ve ekran temizleme. Sağda stories260K için gerçek çıktı: 256 token aynı, hash aynı, ekran, status line, LED'ler ve 7-segment doğru, VGA frame'inde sıfır pixel fark. 18 modelin hepsi geçiyor; toplamda 11 241 token.

39. Random test'ler ve corner case testleri, normal kullanımda ortaya çıkmayan iki RTL bug'ını tespit etti

Normal kullanımla yapılan testler, kullanıcının makul davrandığını varsayar. Gerçek kullanıcılar ise button'lara koşu ortasında basar, reset'e yanlış anda basar. Bunun için iki tür test yazdık. Random test'lerde switch ayarları ve button dizileri random seçiliyor, basışlar koşunun ortasına da denk geliyor, CPU reset de dahil: toplam 655 koşu ve yaklaşık 121 bin token; 32 farklı sampling ayarının 30 ile 32'si kapsandı ve hepsi referansla aynı. Ayrıca 16 corner case elle yazıldı; mesela koşu sırasında basılan button queue'ya girmeli, reset sonrasında yarım kalan koşunun token'ları referansın başlangıcıyla aynı olmalı. Üçüncü test 4-state simülasyon: Verilator her biti 0 ya da 1 kabul ediyor; Icarus ise bilinmeyen değeri, X'i, temsil ediyor ve reset'i unutulmuş register'ları yakalıyor. Bu testler iki gerçek bug buldu. Birincisi, queue'daki bir koşunun başladığı cycle'a denk gelen bir button basışı kayboluyordu. İkincisi, LED'i yakıp söndüren sayacın reset'i yoktu; Verilator her biti 0 ya da 1 saydığı için görünmüyordu, 4-state simülasyonda ise LED X kaldı ve reset button'u bu sayacı sıfırlamıyordu. İkisi de düzeltildi ve bütün testler yeniden geçti.

40. Netlist simülasyonu, RTL doğru olsa bile synthesis araçlarının hata yapabileceğini gösterdi; üç tool bug'ı tespit edildi

RTL'in doğru olması, chip'e giden circuit'in doğru olduğu anlamına gelmiyor; arada RTL'i kapılara çeviren araçlar var ve onlar da hata yapabiliyor. Bu yüzden synthesis aracının ürettiği kapı listesini, yani netlist'i de simüle edip RTL ile pin pin karşılaştırdık. Üç gerçek tool bug'ı bulduk. Birincisi sv2v'de: SystemVerilog'u Verilog'a çeviren bu araç, signed bir array element'ine uygulanan boyut cast'ini unsigned bir ifadeye çeviriyordu. Sonuç olarak attention output'undaki arithmetic shift logical shift'e dönüşüyor ve negatif değerler 127'ye doyuyordu; açık bir signed ile çözdük. İkincisi Yosys'te: iki optimizasyon adımı tek başına doğru, ama art arda çalışınca attention skorundaki dynamic byte select'ini bozuyordu; RTL'i, önce head'in dilimini seçip sonra sabit indeks kullanacak şekilde yeniden yazdık. Üçüncüsü yine Yosys'te: requant multiply'ını DSP block'una map ederken sign extension'ını düşürüyor. Bunu tek başına gösteren küçük bir test yazdık; çözüm olarak Yosys akışında multiplier'ları LUT'lara koyuyoruz. Sonuçta gerçek üst modülün netlist'i, MMCM ve BRAM'ler dahil, üç modelde her pinde ve her cycle'da RTL ile aynı. Verilator ve Vivado SystemVerilog'u doğrudan okuduğu için ilk iki bug'dan etkilenmiyordu; yani bunlar yalnızca open-source akışın bug'larıydı.

41. Synthesis, placement, routing, timing analysis ve bitstream üretimi Vivado olmadan, Mac'te open-source araçlarla yapıldı

Timing'i ölçmek için circuit'in chip'e gerçekten place edilmesi gerekiyor. Xilinx'in aracı Vivado macOS'ta çalışmıyor; bu yüzden tamamen open-source bir akış kurduk. Yosys RTL'i LUT, flip-flop ve BRAM'lerden oluşan bir netlist'e çeviriyor. nextpnr bu netlist'i chip'in fiziksel resource'larına place ediyor, aralarındaki kabloları yönlendiriyor ve chip'in delay modeline göre her path'in delay'ini hesaplıyor; Fmax bu aşamada çıkıyor. Sonra açılacak bütün özellikler FASM adlı bir metin dosyasına

yazılıyor ve Project X-Ray araçları bunu chip'in configuration memory'sinin satırlarına, sonra da .bit dosyasına çeviriyor. Yolda dört sorun çıktı. Mac'in yeni SDK'sını Xcode'un linker'ı okuyamıyordu; Xcode'un kendi SDK'sını kullanarak çözdük. nextpnr'da pin atamasını boş gösteren bir bug bulduk: PACKAGE_PIN değerini dangling bir reference'tan kopyalıyor; kısıt dosyasında doğrudan LOC yazarak aştık. nextpnr 50 megahertz tutmadığında hiçbir rapor yazmadan duruyordu; artık her durumda raporu yazdırıyoruz ve tutmazsa Vivado betiğimizdeki gibi 25 megahertz'te yeniden kuruyoruz. Dördüncüsü daha önce bahsettiğim Yosys DSP bug'ı: bitstream'de multiplier'lar LUT'larda, yani netlist simülasyonunda bit-exact olduğunu gösterdiğimiz netlist. bigram için bütün akış bir dakika sürüyor.

42. makemore'un üç modeli 50 MHz'te timing'i karşılar; transformer'lar 26–31 MHz'te kalır ve 25 MHz'te çalışır

Sonuçlar bu grafikte. Her çubuk, modelin 50 megahertz hedefiyle yapılan placement'ta ulaştığı Fmax; iki dikey çizgi 25 ve 50 megahertz. Mavi çubuklar 50 megahertzi tutan modeller: bigram 71, mlp 66, rnn 67 megahertz. Bunlar en basit üç model; GEMV dışında neredeyse hiç op'ları yok. Turuncu çubuklar tutmayanlar. Transformer'lar 26 ile 31 megahertz arasında kümeleniyor; çünkü hepsi aynı attention ve norm yollarını kullanıyor ve critical path orada. Bunlar 25 megahertz için yeniden kuruldu ve hepsi orada timing'i tutuyor. İki model, bow ve LittleBit'li mlp, 50 megahertzi kıl payı kaçırıyor: 47,7 ve 49,9. Bir bulgu daha var: aynı tasarımın Fmax'ı placement'ın random seed'ine göre oynuyor. bow'un 25 megahertz kurulumu bir seed ile 50,1, ötekiyle 44 megahertz verdi; 50 megahertz denemeleri ise 48,2 ve 47,7. mlp-lb'de 49,4 ve 49,9. Yani bu iki model bir pipeline değişikliği olmadan da başka bir seed ile 50 megahertzi tutabilir. İki TinyStories modeli, ts_gpt2 ve ts_microgpt, yerleşemedi: Yosys bunların memory'sini çok sayıda küçük RAMB18 bloğuna bölüyor, BRAM yerlerinin yüzde 85'i doluyor ve nextpnr'ın placer'ı iki farklı seed ile da 40 dakikada ilerleyemedi. Bunun çözümü memory layout'unu değiştirmek; sonraki adımlar arasında.

43. Critical path, buffer'dan okuma, çarpma ve geri yazmanın tek cycle'da yapılmasından kaynaklanır ve 37 ns sürer

Neden 50 megahertz tutmuyor? nextpnr'ın critical path raporu bunu açıkça gösteriyor. Engine bir op'u yürütürken her cycle'da şunu yapıyor: bir counter'dan vector buffer'ının address'ini hesaplıyor, buffer'ı okuyor, okunan değerlerle çarpma ve toplama yapıyor, sonucu saturate edip buffer'a geri yazıyor; hepsi tek bir cycle'da. Buffer'lar LUT'lardan yapılmış küçük RAM'ler ve okuması asynchronous, yani okuma da aynı path'in parçası. Çarpmalar da Yosys'in DSP bug'ı yüzünden DSP block'larında değil LUT'larda ve uzun carry zincirleri oluşturuyor. stories260K'da en uzun path 36,7 nanosaniye ve 54 adım: bunun yalnızca 9,7 nanosaniyesi logic, 26,9'u routing, yani signal'lerin chip'in bir ucundan diğerine gitmesi. 50 megahertzin period'u 20 nanosaniye; path buna sığmıyor. 25 megahertzin 40 nanosaniyelik period'una ise 3,3 nanosaniye payla sığıyor. Path'in tam yeri modele göre değişiyor: transformer'larda attention'ın head dilimi, binary gpt2'de norm'un output multiply'ı; ama kalıp hep aynı: oku, çarp, yaz, tek cycle'da. Çaresi de belli: okuma ile hesabın arasına bir register koymak. Path ikiye bölünür, op'lar bir cycle gecikir; hedefimiz 50 megahertz.

44. Place and route sonrasında en büyük model bile chip'in logic resource'larının üçte birinden azını kullanır

Place and route sonrasında resource utilization bu. Sayılar nextpnr'dan. LUT sütunu, chip'teki 126 800 LUT yuvasından kullanılanları gösteriyor. En büyük model stories260K'nın binary sürümü bile yüzde 31'de; INT8 stories260K yüzde 27'de. Yani logic tarafında chip'te bolca yer var. Bunun bir ayrıntısı önemli: DSP sütunu hep sıfır, çünkü Yosys'in DSP bug'ı yüzünden bütün multiplier'ları LUT'lara koyduk. Yosys'in DSP kullanan tahmini stories260K için 9 bin LUT ve 66 DSP diyordu; multiplier'lar LUT'a gidince LUT kullanımı birkaç kat artıyor ve critical path da uzuyor. BRAM tarafında sayılar Yosys tahminleriyle birebir aynı: stories260K yüzde 72, binary sürümü yüzde 41. Binary modellerin flip-flop sayısı belirgin şekilde yüksek; 64 lane'li 1-bit unit'in accumulator'ları bunlar.

45. Gerçek hız, clock frequency'sinin token başına cycle sayısına bölümüdür; stories260K saniyede 496 token üretir

Şimdi iki ölçümü birleştirebiliriz. Gerçek hız, clock frequency'sinin token başına cycle sayısına bölümü. Token başına cycle tam kart simülasyonundan geliyor ve sampler ile ekran beklemesini de içeriyor; clock ise placement'ta timing'in tuttuğu frequency. En hızlısı bigram: saniyede 318 bin token, çünkü bir token için yalnızca bir tablo okuyor. makemore'un 50 megahertzde çalışan mlp ve rnn modelleri saniyede 7 500 token civarında. Transformer'lar 25 megahertzde saniyede 2 300 ile 2 900 token arasında. Karpathy'nin stories260K modeli saniyede 496 token yazıyor; 256 token'lık bir hikâye yarım saniyede bitiyor. Low-bit modeller belirgin şekilde hızlı: binary gpt2 saniyede 5 bin, LittleBit'li mlp 13 bin, binary stories260K 893 token. Ölçek vermek için: bir insan saniyede birkaç token okuyabiliyor; kart en yavaş modelde bile okuma hızımızın yüz katından hızlı yazıyor. Bu yüzden karta, üretimi gözle izleyebilmek için saniyede yaklaşık 25 token'lık bir daktilo modu da koyduk. Pipeline ile 50 megahertze çıkabilirsek 25 megahertzdeki bütün sayılar iki katına çıkar.

46. Kalite kaybının tek kaynağı INT8 quantization'dır; FPGA bu kayba hiçbir şey eklemez

Sonuç bölümüne sunumun ana sorusuyla başlamak istiyorum: modelleri FPGA'ye taşırken bir kayıp oluyor mu, oluyorsa nerede? Şekil bunu stories260K üzerinde gösteriyor. Float modelin loss'u 1,400. Modeli INT8 integer programa çevirdiğimizde loss 1,422'ye çıkıyor; kaybın tamamı bu adımda, yani quantization'da oluşuyor ve yaklaşık yüzde 1,6. Names modellerinde bu bedel en fazla 0,0055 nat. İkinci ok FPGA'ye geçiş: RTL ve netlist, integer programla bit-exact; 18 modelin hepsinde tek bir bit bile farklı değil. Dolayısıyla FPGA bu kayba hiçbir şey eklemiyor; FPGA'de ölçtüğümüz kalite, integer programın kalitesiyle aynı. Alttaki tablo hız. Aynı modeli Karpathy'nin kendi C koduyla, run.c ile, Apple M1 CPU'da tek thread'de ve en iyi derleyici ayarıyla çalıştırdık: saniyede 14 397 token. Yani mutlak hızda dizüstü bilgisayar 29 kat önde; bunun sebebi clock'un 128 kat yüksek olması. Token başına harcanan cycle'a bakınca tablo tersine dönüyor: CPU token başına yaklaşık 222 273 cycle harcıyor, FPGA 50 397; yani clock başına FPGA 4,4 kat daha fazla iş yapıyor. Pipeline ile 50 megahertze çıkıldığında FPGA'nin hızı iki katına çıkar. Sağdaki kutu, henüz iddia etmediğimiz şeyler: kart olmadığı için enerji ölçmedik, Fmax açık kaynak timing modeline dayanıyor ve DSP'ler kullanılmıyor; hızdaki en büyük pay pipeline eklemekte.

47. Kalite ve hız birlikte değerlendirildiğinde names'te en iyi loss gru'da, en iyi denge rnn ve mlp'dedir

Son olarak bütün aileyi iki eksende karşılaştıralım. Dikey eksen kalite: kartın çalıştırdığı integer programın test loss'u; yukarı gittikçe daha iyi. Yatay eksen hız: kartta saniyede üretilen token, logarithmic scale; sağa gittikçe daha hızlı. Yani sağ üst köşe en iyi. Mavi noktalar makemore modelleri, turuncular transformer'lar, kareler low-bit modeller. Bu ölçekte, isim üretiminde, en iyi loss gru'da; ama gru 25 megahertzde çalıştığı için saniyede 3 800 token. rnn ve mlp hem iyi loss'a sahip hem de 50 megahertzi tuttukları için saniyede 7 500 token civarında; bu ölçekte en iyi denge onlar. Transformer'lar sol üstte toplanıyor: loss'ları iyi ama norm ve attention yüzünden hem daha çok cycle harcıyorlar hem de 25 megahertzde kalıyorlar. Binary gpt2 transformer'ları hız ekseninde neredeyse iki kat sağa taşıyor, loss'ta küçük bir bedelle. LittleBit'li mlp en hızlısı, saniyede 13 bin token, ama loss'ta en büyük bedeli ödüyor. bigram ölçeğin çok dışında: saniyede 318 bin token, ama loss 2,45 ile en kötüsü. Bu grafik isim gibi kısa diziler için; TinyStories gibi uzun context'li görevlerde attention'ın payı büyüdüğü için transformer'ların durumu değişiyor.

48. Kullanılan tekniklerin çoğu literatürde bilinmektedir; çalışmanın özgünlüğü birleşimde, ölçüm düzeneğinde ve verification'dadır

Bu projenin özgün tarafı ne? Dürüst cevap şu: tek tek tekniklerin çoğu literatürde var; özgün olan, onları bir araya getirişimiz, kurduğumuz ölçüm düzeneği ve verification. Soldaki beş madde bizim katkımız. Birincisi ve bence en savunulabiliri: Karpathy'nin dokuz mimarisini eşit parametre bütçesi, aynı veri, aynı integer arithmetic ve aynı hardware ile karşılaştırdık; böylece RMSNorm, RoPE, GQA, SwiGLU ya da QK-norm gibi her mimari fikrin cycle, resource, Fmax ve kalite bedeli ayrı ayrı görünüyor. Taramamızda eşit bütçede her yeniliğin FPGA maliyetini raporlayan bir çalışma bulamadık. İkincisi: RNN'den nanochat'e kadar bu kadar farklı on modeli, Karpathy'nin gerçek checkpoint'i dahil, tek bir microcoded engine'de ve compiler'ıyla birlikte, hepsi bit-exact çalıştırmak. Literatürdeki FPGA language model çalışmaları genellikle tek bir modeli hedefliyor ve normalization'ı çoğu zaman floating point'te bırakıyor. Üçüncüsü: kart bağımsız; benzer çalışmaların çoğu bir host CPU'ya dayanıyor, bizde sampling bile integer ve FPGA'de. Dördüncüsü: her integer işlemin range'ini ispatladık ve pin seviyesinde bit-exact doğruladık; bu süreçte dört gerçek tool bug'ı bulduk. Beşincisi: hocamın verdiği LittleBit'i küçük ölçekte inceledik; scale maliyetinin dar layer'lardaki sınırını formülle ve ölçümle gösterdik ve hardware'de binary ile aynı unit'te çalıştırdık. Sağdaki sütun literatürden aldıklarımız: requant, integer softmax ve LayerNorm fikirleri, output-stationary dataflow, komutla programlanan engine geleneği, Larq'ın quantizer'ları ve LittleBit'in kendisi. En yakın çalışmaları da belirtmek istiyorum: HLSTransform, Karpathy'nin llama2.c modelinin 110 milyon parametreli sürümünü bir FPGA'de çalıştırıyor, ancak weight'ler external memory'de ve sampling host bilgisayarda yapılıyor. TALOS-V2, Karpathy'nin microGPT'sini SystemVerilog'da çalıştıran open-source bir proje; tek bir model ve JTAG üzerinden bilgisayarla sürülüyor. COBRA ise 1-bit bir matrix unit kullanıyor, ancak metin üretmiyor. Bu değerlendirme Ekim 2026 sonunda yaptığımız literatür taramasına dayanıyor.

49. Çalışmanın sınırları: kanıtlanan ve henüz kanıtlanmayan noktalar

Sonuçları değerlendirmeden önce sınırları açıkça söylemek istiyorum. Birincisi, fiziksel kart testi yapmadık; bu bir karardı. Kanıtımız simülasyon ve timing analysis. Burada tek boşluk şu:

placement'tan sonraki son circuit'i simüle edemiyoruz, çünkü nextpnr onu simüle edilebilir Verilog olarak yazamıyor; o aşama için timing analysis'e güveniyoruz. İkincisi, ölçtüğümüz Fmax yaklaşık. nextpnr'ın kullandığı delay modeli Project X-Ray topluluğunun chip'ten çıkardığı bir model; Vivado'nun sonucu farklı olabilir. Ayrıca multiplier'lar LUT'larda olduğu için bu circuit Vivado'nun DSP'li circuit'inden daha yavaş ve daha büyük. Üçüncüsü kapasite: chip'in 607 kilobyte'lık BRAM'i yaklaşık 300 bin parametreye kadar yetiyor. nanoGPT'nin ya da llm.c'nin hazır eğitilmiş modelleri on milyonlarca parametre; onlar için kartın DDR2 memory'sinden weight okuyan farklı bir mimari gerekir. Dördüncüsü eksik parçalar: iki TinyStories modeli, ts_gpt2 ve ts_microgpt, memory'leri çok sayıda küçük BRAM bloğuna bölündüğü için yerleşemedi; ilk sunumun konusu olan minGPT'nin sıralama demosu bu kart tasarımında yok, eski hızlandırıcıdaydı; Larq'ın gerçek paketiyle sayısal karşılaştırma yapmadık, çünkü TensorFlow kurmadık; LittleBit'in iki path'li sürümü de hardware'de yok, yalnızca tek path'li.

50. Sonraki adımlar öncelik sırasıyla hız, verification'ın tamamlanması ve kapsamın genişletilmesidir

Sonraki adımları öncelik sırasıyla sıraladım. Birincisi hız: engine'de buffer okuması ile hesabın arasına bir register koymak. Bu, critical path'i ikiye bölüyor ve hedefimiz 50 megahertz; bu da token/s'de iki kat demek. Bu bir RTL değişikliği olduğu için 18 modelin bütün simülasyonları yeniden geçmeli. İkincisi verification'ın son boşluğu: placement'a giren netlist'i, gerçek clock ayarı ve gerçek debounce süreleriyle, 18 modelin hepsinde simüle etmek. Üçüncüsü DSP block'ları: Yosys'teki bug'ı geliştiricilere raporlamak ve bug'ın görüldüğü requant multiply'ı dışındaki multiplier'ları yeniden DSP'ye koymak. Bu hem LUT sayısını düşürür hem Fmax'ı artırır. Aynı adımda place edilemeyen iki TinyStories modelinin memory'lerini, Yosys onları küçük block'lara bölmeyecek şekilde yeniden düzenlemek gerekiyor. Dördüncüsü kapsam: ilk sunumun konusu olan minGPT'nin sıralama demosunu 19. model olarak kart tasarımına eklemek; ekranda girdi dizisini ve sıralanmış çıktıyı göstermek. Beşincisi low-bit tarafı: stories260K için daha uzun bir binary QAT, çünkü loss son adımlarda hâlâ düşüyordu; ve LittleBit'in iki path'li sürümünü hardware'de denemek.

51. son

Toparlamak için dört cümle. Birincisi: Karpathy'nin dokuz language model mimarisi ve hazır eğitilmiş stories260K modeli tek bir microcoded integer engine'de çalışıyor; bir modelden ötekine geçerken circuit değişmiyor, yalnızca BRAM'deki microcode ve weight'ler değişiyor. İkincisi: kart hiçbir bilgisayara bağlı olmadan çalışıyor; generation loop, integer sampling, token'ların metne çevrilmesi ve VGA ekranı FPGA'nin içinde. Kartın tamamının simülasyonunda 18 modelin hepsi Python'daki referansla bit bit aynı sonuç veriyor. Üçüncüsü: hocamın önerdiği yöntemlerden binary weight'ler 1,7 ile 3,6 kat hız ve yarı memory getiriyor; LittleBit ise küçük modellerde scale maliyeti yüzünden ancak geniş layer'larda binary'yi geçiyor. Dördüncüsü: Vivado olmadan, open-source araçlarla Mac'te bitstream ürettik ve timing'i ölçtük. 16 model yerleşti; makemore'un en basit üç modeli 50 megahertzde, diğerleri 25 megahertzde timing'i tutuyor. Engine'e pipeline ekleyerek hedefimiz hepsini 50 megahertze çıkarmak. Teşekkür ederim; sorularınızı bekliyorum.